

copia_01_profiling_empresas_profesional_con_matching - copia

April 12, 2026

1 Data Profiling e Integración de Fuentes para Clustering de Empresas

1.1 Objetivo

Este cuaderno documenta el proceso de análisis, validación e integración de dos fuentes de datos empresariales con el objetivo de construir un dataset adecuado para técnicas de clustering.

1.2 Fuentes de datos

- Dataset 1: Leads provenientes de CRM
- Dataset 2: Registro de horas trabajadas por empresa

1.3 Enfoque

Se sigue una metodología basada en:

1. Data profiling
2. Validación de hipótesis de integración
3. Evaluación de consistencia entre fuentes
4. Toma de decisiones basada en evidencia
5. Recomendación de siguiente paso (staging y estrategia de integración)

1.4 1. Imports y configuración

```
[12]: import pandas as pd
import numpy as np
import re
import unicodedata
from pathlib import Path

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

1.5 2. Configuración de archivos

```
[13]: LEADS_FILE = 'leads.xlsx'
HORAS_FILE = 'proyectos_empresa.xlsx'

N_FILAS_PRUEBA = None
```

1.6 3. Función de carga de datos

```
[14]: def leer_archivo(path, nrow=None):
    path = Path(path)
    suffix = path.suffix.lower()

    if suffix == '.csv':
        return pd.read_csv(path, nrow=nrow)
    elif suffix in ['.xlsx', '.xls']:
        return pd.read_excel(path, nrow=nrow)
    else:
        raise ValueError(f'Formato no soportado: {suffix}')
```

1.7 4. Carga de datasets

```
[15]: df_leads = leer_archivo(LEADS_FILE, nrow=N_FILAS_PRUEBA)
df_horas = leer_archivo(HORAS_FILE, nrow=N_FILAS_PRUEBA)

print('Leads:', df_leads.shape)
print('Horas:', df_horas.shape)
```

Leads: (440, 15)

Horas: (412, 18)

```
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\env\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
warn("Workbook contains no default style, apply openpyxl's default")
```

1.8 4.b Perfilado rápido (calidad y esquema)

```
[16]: def _profile_table(df: pd.DataFrame) -> pd.DataFrame:
    out = pd.DataFrame({
        'columna': df.columns,
        'dtype': [str(df[c].dtype) for c in df.columns],
        'n_null': [int(df[c].isna().sum()) for c in df.columns],
        'pct_null': [float(df[c].isna().mean() * 100) for c in df.columns],
        'n_unique': [int(df[c].nunique(dropna=True)) for c in df.columns],
    })
    return out.sort_values(['pct_null', 'n_unique'], ascending=[False, True]).
    ↪reset_index(drop=True)
```

```

def _normalize_for_compare(value) -> str:
    if value is None:
        return ''
    if isinstance(value, float) and np.isnan(value):
        return ''
    s = str(value).strip()
    if not s:
        return ''
    s = unicodedata.normalize('NFKD', s)
    s = ''.join(ch for ch in s if not unicodedata.combining(ch))
    s = s.upper()
    s = re.sub(r'[^A-Z0-9 ]+', ' ', s)
    s = re.sub(r'\s+', ' ', s).strip()
    return s

def _company_quality(df: pd.DataFrame, col: str, label: str) -> None:
    if col not in df.columns:
        print(f'[{label}] Columna no encontrada: {col}')
        return
    s = df[col]
    s_str = s.astype('string')
    blank = s_str.isna() | (s_str.str.strip() == '')
    print(f'\n[{label}] Calidad de {col}:')
    print('- filas:', len(df))
    print('- n_null:', int(s.isna().sum()))
    print('- n_blank (null o vacío):', int(blank.sum()))
    print('- n_unique (no-null):', int(s.nunique(dropna=True)))
    print('- n_duplicadas (por valor no-null):', int(s.dropna().duplicated().
↪sum()))

    top = (s_str.fillna('')
            .map(lambda x: x.strip())
            .replace('', pd.NA)
            .dropna()
            .value_counts()
            .head(15)
            .reset_index())
    if len(top):
        top.columns = [col, 'freq']
        display(top)

    # Normalización rápida para estimar colisiones
    norm = s_str.map(_normalize_for_compare)
    norm = norm.replace('', pd.NA).dropna()
    if len(norm):
        print('- n_unique_norm:', int(norm.nunique()))

```

```

        print('- colisiones_por_norm (valores distintos que caen al mismo norm):
↳', int(norm.duplicated().sum()))

def _exact_match_summary() -> None:
    if 'Company' not in df_leads.columns or 'EMPRESA' not in df_horas.columns:
        print('\n[JOIN] No se puede calcular resumen: falta Company o EMPRESA.')
        return

    leads_raw = (df_leads['Company'].astype('string').fillna('').map(lambda x:
↳x.strip()))
    horas_raw = (df_horas['EMPRESA'].astype('string').fillna('').map(lambda x:
↳x.strip()))
    leads_raw = leads_raw.replace('', pd.NA).dropna()
    horas_raw = horas_raw.replace('', pd.NA).dropna()

    raw_matches = sorted(set(leads_raw).intersection(set(horas_raw)))
    print('\n[JOIN] Coincidencias exactas (sin normalizar):', len(raw_matches))
    if len(raw_matches):
        print(' Muestra:', raw_matches[:20])

    leads_norm = leads_raw.map(_normalize_for_compare).replace('', pd.NA).
↳dropna()
    horas_norm = horas_raw.map(_normalize_for_compare).replace('', pd.NA).
↳dropna()
    norm_matches = sorted(set(leads_norm).intersection(set(horas_norm)))
    print('[JOIN] Coincidencias exactas (normalizadas):', len(norm_matches))
    if len(norm_matches):
        print(' Muestra:', norm_matches[:20])

print('--- Perfilado LEADS ---')
display(_profile_table(df_leads))

print('\n--- Perfilado HORAS ---')
display(_profile_table(df_horas))

# Calidad de las llaves candidatas para join
_company_quality(df_leads, 'Company', 'LEADS')
_company_quality(df_horas, 'EMPRESA', 'HORAS')

# Evidencia cuantitativa: ¿hay intersección exacta?
_exact_match_summary()

```

--- Perfilado LEADS ---

	columna	dtype	n_null	pct_null	n_unique
0	First Name	float64	440	100.000000	0
1	Last Name	float64	440	100.000000	0
2	Email	float64	440	100.000000	0

3	Phone	float64	440	100.000000	0
4	Mobile	float64	440	100.000000	0
5	Website	float64	440	100.000000	0
6	No. of Employees	float64	440	100.000000	0
7	Annual Revenue	float64	440	100.000000	0
8	Linkedin	float64	440	100.000000	0
9	País.	str	415	94.318182	8
10	Lead Source	str	408	92.727273	1
11	Industry	str	408	92.727273	4
12	Cargo	str	87	19.772727	290
13	Interés	str	0	0.000000	1
14	Company	str	0	0.000000	330

--- Perfilado HORAS ---

	columna	dtype	n_null	pct_null	n_unique
0	AVANCE_REAL	float64	412	100.000000	0
1	AVANCE_ESTIMADO	float64	412	100.000000	0
2	ID_COL_RESPONSABLE	float64	379	91.990291	7
3	FACTURACION	float64	235	57.038835	120
4	HORAS_ESTIMADAS	float64	187	45.388350	107
5	HORAS_EJECUTADAS_FACTURABLES	float64	26	6.310680	314
6	HORAS_EJECUTADAS	float64	20	4.854369	319
7	FECHA_CORTE	datetime64[us]	19	4.611650	5
8	EN_EJECUCION	float64	4	0.970874	2
9	MOSTRAR_LISTAS	int64	0	0.000000	2
10	LUGAR_POR_DEFECTO	int64	0	0.000000	2
11	TIPO_ALCANCE	str	0	0.000000	3
12	OCUPACION	str	0	0.000000	4
13	ID_ESP	int64	0	0.000000	15
14	EMPRESA	str	0	0.000000	120
15	ID_EMP	int64	0	0.000000	120
16	NOMBRE	str	0	0.000000	372
17	ID_PRO	int64	0	0.000000	412

[LEADS] Calidad de Company:

- filas: 440
- n_null: 0
- n_blank (null o vacío): 0
- n_unique (no-null): 330
- n_duplicadas (por valor no-null): 110

	Company	freq
0	GRUPO ALEN	10
1	PEPSICO	7
2	GRUPO BIMBO	7
3	WALMART	7
4	BACARDI	5

5	LAMOSA	5
6	CASA CUERVO	5
7	SEGUROS MONTERREY NEW YORK LIFE	5
8	UNILEVER	4
9	AIG	4
10	NOVARTIS	4
11	ASTRAZENECA	3
12	SANOFI	3
13	RAPPI	3
14	GNP SEGUROS	3

- n_unique_norm: 328

- colisiones_por_norm (valores distintos que caen al mismo norm): 112

[HORAS] Calidad de EMPRESA:

- filas: 412

- n_null: 0

- n_blank (null o vacío): 0

- n_unique (no-null): 120

- n_duplicadas (por valor no-null): 292

	EMPRESA	freq
0	Corporacion GPF	66
1	Corporación Maresa	22
2	FPA	21
3	Veolia Latam	21
4	Aseguradora del Sur	20
5	Intaco	12
6	La Fabril	10
7	NOVA Ecuador	10
8	Telefónica EC	10
9	Millicom - Telefónica PA, NI	9
10	Adium	8
11	Seguros del Pichincha	7
12	Veolia China	7
13	Induglob	6
14	Veolia Argentina	6

- n_unique_norm: 119

- colisiones_por_norm (valores distintos que caen al mismo norm): 293

[JOIN] Coincidencias exactas (sin normalizar): 0

[JOIN] Coincidencias exactas (normalizadas): 0

1.9 5. Validación de hipótesis de integración

1.9.1 Hipótesis

Se plantea que:

Las empresas presentes en el dataset de leads deberían coincidir con las empresas presentes en el dataset de horas trabajadas.

1.9.2 Objetivo

Validar si es posible realizar un join confiable entre ambas fuentes.

1.10 6. Evaluación detallada (coincidencias exactas normalizadas y aproximadas)

El resumen de la sección 4.b (Perfilado rápido) puede ampliarse con una rutina más detallada de normalización y similitud. Esta sección se conserva porque aporta evidencia técnica adicional:

- normaliza nombres con eliminación de acentos,
- reduce ruido por sufijos legales comunes,
- calcula coincidencias exactas después de normalizar,
- y genera sugerencias por similitud para evidenciar que no existen matches confiables.

Esto respalda formalmente la decisión de **no forzar un join** entre ambas fuentes usando solo el nombre de la empresa.

```
[17]: import re
import unicodedata
import pandas as pd

def _strip_accents(s: str) -> str:
    return ''.join(ch for ch in unicodedata.normalize('NFKD', s) if not
↳ unicodedata.combining(ch))

def _normalize_name(s: str) -> str:
    if s is None:
        return ""
    s = str(s).strip()
    s = _strip_accents(s)
    s = s.upper()
    # quitar caracteres raros, dejar letras/números/espacios
    s = re.sub(r"[^A-Z0-9 ]+", " ", s)
    # quitar sufijos legales comunes
    s = re.sub(r"\b(SA|S A|S\.A|S\.A\.
↳ S|SAS|LTDA|CIA|CORP|INC|LLC|DE|DEL|LA|EL)\b", " ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s

# Tomar listas directamente desde los DataFrames (evita dependencia de
↳ secciones eliminadas)
if 'Company' not in df_leads.columns or 'EMPRESA' not in df_horas.columns:
    print("No se puede comparar: falta Company o EMPRESA en los dataframes.")
else:
    companies = (df_leads['Company'].astype('string').fillna(''))
```

```

        .map(lambda x: x.strip())
        .replace('', pd.NA)
        .dropna()
        .unique()
        .tolist()
empresas = (df_horas['EMPRESA'].astype('string').fillna('')
            .map(lambda x: x.strip())
            .replace('', pd.NA)
            .dropna()
            .unique()
            .tolist())

if not companies or not empresas:
    print("No hay valores suficientes para comparar (listas vacías).")
else:
    comp_norm = {c: _normalize_name(c) for c in companies}
    emp_norm = {e: _normalize_name(e) for e in empresas}

    # 1) Coincidencias exactas después de normalizar
    inv_emp = {}
    for e, ne in emp_norm.items():
        inv_emp.setdefault(ne, []).append(e)

    exact_matches = []
    for c, nc in comp_norm.items():
        for e in inv_emp.get(nc, []):
            exact_matches.append({
                "Company": c,
                "EMPRESA": e,
                "tipo": "exact_norm",
                "score": 100
            })

    df_exact = pd.DataFrame(exact_matches)
    print(f"Coincidencias EXACTAS (tras normalizar): {len(df_exact)}")
    if len(df_exact):
        display(df_exact.sort_values(['Company', 'EMPRESA']).
        ↪reset_index(drop=True))

    # 2) Coincidencias por similitud (fuzzy). Si está rapidfuzz, mejor; si
    ↪no, usa difflib.
    rows = []
    try:
        from rapidfuzz import process, fuzz
        scorer = fuzz.token_set_ratio
        emp_choices = list(emp_norm.keys())
        for c, nc in comp_norm.items():

```



```

        best = process.extractOne(nc, emp_choices, scorer=scorer)
        if best is None:
            continue
        best_norm, score, _ = best
        e_orig = inv_emp.get(best_norm, [None])[0]
        rows.append({
            "Company": c,
            "Company_norm": nc,
            "EMPRESA_best": e_orig,
            "EMPRESA_norm": best_norm,
            "score": float(score),
        })
    engine = "rapidfuzz"
except Exception:
    from difflib import SequenceMatcher

    def ratio(a, b):
        return SequenceMatcher(None, a, b).ratio() * 100

    emp_choices = list(emp_norm.keys())
    for c, nc in comp_norm.items():
        best_norm = None
        best_score = -1
        for en in emp_choices:
            sc = ratio(nc, en)
            if sc > best_score:
                best_score = sc
                best_norm = en
        e_orig = inv_emp.get(best_norm, [None])[0] if best_norm is not None
        ↪None else None
        rows.append({
            "Company": c,
            "Company_norm": nc,
            "EMPRESA_best": e_orig,
            "EMPRESA_norm": best_norm,
            "score": float(best_score),
        })
    engine = "difflib"

df_fuzzy = pd.DataFrame(rows).sort_values('score', ascending=False).
↪reset_index(drop=True)
print(f"\nMotor de similitud: {engine}")
print("Sugerencia: revisar coincidencias con score alto (p.ej. >= 80).")

display(df_fuzzy.head(30))
umbral = 80
df_high = df_fuzzy[df_fuzzy['score'] >= umbral]

```

```
print(f"\nCoincidencias sugeridas con score >= {umbral}:\n")
↳len(df_high)})")
display(df_high.reset_index(drop=True))
```

Coincidencias EXACTAS (tras normalizar): 0

Motor de similitud: rapidfuzz

Sugerencia: revisar coincidencias con score alto (p.ej. >= 80).

	Company	Company_norm	EMPRESA_best	EMPRESA_norm	score
0	OSRAM	OSRAM	CORSAM	CORSAM	72.727273
1	FEMSA	FEMSA	FADESA	FADESA	72.727273
2	SMI	SMI	BMI	BMI	66.666667
3	BIC	BIC	BAC	BAC	66.666667
4	UCB	UCB	UPC	UPC	66.666667
5	CBC	CBC	BAC	BAC	66.666667
6	Chronos	CHRONOS	CONSEP	CONSEP	61.538462
7	LACOSTE	LACOSTE	CONSEP	CONSEP	61.538462
8	BACARDI	BACARDI	BANRED	BANRED	61.538462
9	BACHOCO	BACHOCO	BAC	BAC	60.000000
10	PEPSICO	PEPSICO	SIC	SIC	60.000000
11	Abbvie	ABBVIE	AVIS	AVIS	60.000000
12	Onapis	ONAPIS	AVIS	AVIS	60.000000
13	BALL	BALL	BAC	BAC	57.142857
14	PROESSA	PROESSA	PROPHAR	PROPHAR	57.142857
15	PPG	PPG	KPMG	KPMG	57.142857
16	BECLE SAB	BECLE SAB	CEPSA	CEPSA	57.142857
17	AIG	AIG	AVIS	AVIS	57.142857
18	URBI	URBI	BMI	BMI	57.142857
19	Pozuelo	POZUELO	PUCE	PUCE	54.545455
20	ONTEX	ONTEX	CONSEP	CONSEP	54.545455
21	TRANE	TRANE	BANRED	BANRED	54.545455
22	ESACERO S.A.	ESACERO	ESPE	ESPE	54.545455
23	Parauco	PARAUCO	PUCE	PUCE	54.545455
24	Arcor	ARCOR	CORSAM	CORSAM	54.545455
25	COMEX	COMEX	CONSEP	CONSEP	54.545455
26	ICONN	ICONN	CONSEP	CONSEP	54.545455
27	Confiteca	CONFITECA	CONSEP	CONSEP	53.333333
28	CXO Forum	CXO FORUM	CORSAM	CORSAM	53.333333
29	EUROFARMA	EUROFARMA	PROPHAR	PROPHAR	50.000000

Coincidencias sugeridas con score >= 80: 0

Empty DataFrame

Columns: [Company, Company_norm, EMPRESA_best, EMPRESA_norm, score]

Index: []

1.10.1 Interpretación técnica de esta validación

Si esta sección produce:

- **0 coincidencias exactas tras normalización**, y
- **0 coincidencias con score alto**,

entonces existe evidencia suficiente para afirmar que ambas fuentes no pueden integrarse de forma confiable mediante el nombre de la empresa.

Este resultado no representa una falla del código, sino un hallazgo del análisis de calidad e integración de datos.

Nota de organización: la generación de **data cruda normalizada** (staging) y el **export** a CSV se movieron a `02_data_cleaning/01_raw_normalizado_export.ipynb` para no mezclar ingesta/profiling con cleaning.

1.11 7. Resultado de la validación

No se encontraron coincidencias exactas entre las empresas de ambas fuentes.

1.11.1 Interpretación

Esto indica que:

- Las fuentes no están alineadas
- No es posible realizar un join directo confiable
- Existe inconsistencia en la representación de entidades

1.11.2 Conclusión

Se rechaza la hipótesis de integración directa.

1.12 8. Próximos pasos (según evidencia)

- No forzar join `Company` `EMPRESA` por nombre: el profiling muestra que no hay coincidencias confiables.
- Preparar staging normalizado para BD/joins posteriores en `02_data_cleaning/01_raw_normalizado_export.ipynb` (export a CSV).
- Definir estrategia alternativa de integración (p.ej. catálogo unificado de empresas / matching asistido / llaves adicionales).

1.13 9. Universo unificado de empresas (LEADS HORAS) y verificación contra SCVS

Objetivo: como no hay coincidencias exactas entre ambas fuentes, construimos el **universo unificado** de empresas (a partir de ambas) y evaluamos cobertura en una fuente externa (SCVS / Superintendencia de Compañías).

- Esta sección **no exporta** CSV; solo genera dataframes y métricas para el análisis.

```
[18]: # === 9.a Limpieza LEADS (Company) ===
```

```

# Resultado esperado: DataFrame `leads_companies_clean` con Company raw +
↳ normalizada (distinct por normalizada)

from pathlib import Path
import re
import unicodedata
import numpy as np
import pandas as pd

def _strip_accents(text: str) -> str:
    return "".join(
        ch for ch in unicodedata.normalize("NFKD", text)
        if not unicodedata.combining(ch)
    )

def normalize_company_name(value) -> str:
    """Normaliza nombres de empresa para comparación (no para mostrar)."""
    if value is None:
        return ""
    if isinstance(value, float) and np.isnan(value):
        return ""

    s = str(value).strip()
    if not s:
        return ""

    s = _strip_accents(s)
    s = s.upper()
    s = re.sub(r"^[A-Z0-9 ]+", " ", s)

    # Quitar sufijos/palabras frecuentes que distorsionan el matching
    s = re.sub(
        r"\b(SA|S*s*A|S\.*A\.*|S\.*A\.*S\.*|SAS|LTDA|CIA|C\.*IA\.*|
↳ |COMPANIA|COMPAÑIA|CORP|INC|LLC|C\.*L\.*|C\.*LTDA\.*|\&|Y)\b",
        " ",
        s,
    )
    s = re.sub(r"\b(DE|DEL|LA|EL|LOS|LAS)\b", " ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s

if "df_leads" not in globals():
    raise NameError("No existe `df_leads` en memoria. Ejecuta la sección de
↳ carga de datasets (Sección 4).")

```

```

if "Company" not in df_leads.columns:
    raise KeyError("La columna `Company` no existe en `df_leads`.")

_company_raw = (
    df_leads["Company"]
    .astype("string")
    .fillna("")
    .map(lambda x: x.strip())
    .replace("", pd.NA)
    .dropna()
)

leads_companies_clean = (
    pd.DataFrame({"Company_raw": _company_raw})
    .assign(Company_norm=lambda d: d["Company_raw"].map(normalize_company_name))
    .query("Company_norm != ''")
    .drop_duplicates(subset=["Company_norm"], keep="first")
    .sort_values("Company_norm")
    .reset_index(drop=True)
)

print("[LEADS] Filas originales (no vacías):", int(_company_raw.shape[0]))
print("[LEADS] Empresas distinct (por Company_norm):",
      ↪int(leads_companies_clean.shape[0]))

display(leads_companies_clean.head(20))

# --- Export CSV (para descargar desde VS Code) ---
# Guardar en outputs/ relativo al cwd (si estás dentro de ↪
      ↪01_data_ingestion_enrichment),
# o en 01_data_ingestion_enrichment/outputs si el cwd es el root del repo.
OUT_DIR = Path("outputs") if Path.cwd().name == "01_data_ingestion_enrichment" ↪
      ↪else (Path("01_data_ingestion_enrichment") / "outputs")
OUT_DIR.mkdir(parents=True, exist_ok=True)

out_path = OUT_DIR / "leads_companies_clean.csv"
leads_companies_clean.to_csv(out_path, index=False, encoding="utf-8-sig")
print(f"[LEADS] CSV guardado en: {out_path.resolve()}")

```

[LEADS] Filas originales (no vacías): 440

[LEADS] Empresas distinct (por Company_norm): 326

	Company_raw	Company_norm
0	7-ELEVEN MEXICO	7 ELEVEN MEXICO
1	ABBOTT	ABBOTT
2	Abbvie	ABBVIE
3	ACCO BRANDS	ACCO BRANDS

4	ACH FOOD COMPANIES, INC	ACH FOOD COMPANIES
5	ACTINVER	ACTINVER
6	ADAMANTINE	ADAMANTINE
7	AFP Genesis	AFP GENESIS
8	AIG	AIG
9	Akros	AKROS
10	ALCIONE MX	ALCIONE MX
11	AlimentosAlConsumidor	ALIMENTOSALCONSUMIDOR
12	ALLIANZ MÉXICO	ALLIANZ MEXICO
13	ALPEZZI CHOCOLATE	ALPEZZI CHOCOLATE
14	Alpina	ALPINA
15	Alvarez Barba S.A	ALVAREZ BARBA
16	AMBROSIA	AMBROSIA
17	ANHEUSER -BUSCH INBEV	ANHEUSER BUSCH INBEV
18	La Anita	ANITA
19	APOTEX	APOTEX

[LEADS] CSV guardado en: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\leads_companies_clean.csv

```
[19]: # === 9.b Limpieza HORAS (EMPRESA) ===
# Resultado esperado: DataFrame `horas_empresas_clean` con EMPRESA raw +
↳ normalizada (distinct por normalizada)

from pathlib import Path
import pandas as pd

if "df_horas" not in globals():
    raise NameError("No existe `df_horas` en memoria. Ejecuta la sección de
↳ carga de datasets (Sección 4).")

if "EMPRESA" not in df_horas.columns:
    raise KeyError("La columna `EMPRESA` no existe en `df_horas`.")

_empresa_raw = (
    df_horas["EMPRESA"]
    .astype("string")
    .fillna("")
    .map(lambda x: x.strip())
    .replace("", pd.NA)
    .dropna()
)

horas_empresas_clean = (
    pd.DataFrame({"EMPRESA_raw": _empresa_raw})
    .assign(EMPRESA_norm=lambda d: d["EMPRESA_raw"].map(normalize_company_name))
    .query("EMPRESA_norm != ''")
    .drop_duplicates(subset=["EMPRESA_norm"], keep="first")
)
```

```

        .sort_values("EMPRESA_norm")
        .reset_index(drop=True)
    )

print("[HORAS] Filas originales (no vacías):", int(_empresa_raw.shape[0]))
print("[HORAS] Empresas distinct (por EMPRESA_norm):", int(horas_empresas_clean.
↳shape[0]))

display(horas_empresas_clean.head(20))

# --- Export CSV (para descargar desde VS Code) ---
OUT_DIR = Path("outputs") if Path.cwd().name == "01_data_ingestion_enrichment"
↳else (Path("01_data_ingestion_enrichment") / "outputs")
OUT_DIR.mkdir(parents=True, exist_ok=True)

out_path = OUT_DIR / "horas_empresas_clean.csv"
horas_empresas_clean.to_csv(out_path, index=False, encoding="utf-8-sig")
print(f"[HORAS] CSV guardado en: {out_path.resolve()}")

# --- CSV compilado de empresas (LEADS HORAS) ---
if "leads_companies_clean" in globals() and leads_companies_clean is not None
↳and not leads_companies_clean.empty:
    leads_u = (
        leads_companies_clean
        .rename(columns={"Company_raw": "name_raw", "Company_norm":
↳"name_norm"})
        .assign(source="LEADS")
        [["source", "name_raw", "name_norm"]]
    )
    horas_u = (
        horas_empresas_clean
        .rename(columns={"EMPRESA_raw": "name_raw", "EMPRESA_norm":
↳"name_norm"})
        .assign(source="HORAS")
        [["source", "name_raw", "name_norm"]]
    )

    universe_all = pd.concat([leads_u, horas_u], ignore_index=True)
    universe_distinct = (
        universe_all
        .groupby("name_norm", as_index=False)
        .agg(
            name_raw=("name_raw", "first"),
            sources=("source", lambda s: ",".join(sorted(set(map(str, s))))),
            n_rows=("source", "size"),
        )
        .sort_values("name_norm")

```

```

        .reset_index(drop=True)
    )

    out_universe = OUT_DIR / "empresas_universe_compilado.csv"
    universe_distinct.to_csv(out_universe, index=False, encoding="utf-8-sig")
    print(f"[UNIVERSO] CSV compilado guardado en: {out_universe.resolve()}")
    display(universe_distinct.head(20))
else:
    print("[UNIVERSO] Nota: no existe `leads_companies_clean` (ejecuta primero_
↪9.a) para generar el compilado LEADS HORAS.")

```

[HORAS] Filas originales (no vacías): 412

[HORAS] Empresas distinct (por EMPRESA_norm): 119

	EMPRESA_raw	EMPRESA_norm
0	3dpharma	3DPHARMA
1	Adium	ADIUM
2	Almexa	ALMEXA
3	Alper Seguros	ALPER SEGUROS
4	Arauco	ARAUCO
5	Aseguradora del Sur	ASEGURADORA SUR
6	Asesoría y Control	ASESORIA CONTROL
7	AutoShare	AUTOSHARE
8	AVIS	AVIS
9	BAC	BAC
10	Banco del Austro	BANCO AUSTRO
11	Banco Financiero	BANCO FINANCIERO
12	BANRED	BANRED
13	Bekaert	BEKAERT
14	Bemis	BEMIS
15	Bitso	BITSO
16	Bloomin Brands	BLOOMIN BRANDS
17	BMI	BMI
18	Boston BSCI	BOSTON BSCI
19	CEPSA	CEPSA

[HORAS] CSV guardado en: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\horas_empresas_clean.csv

[UNIVERSO] CSV compilado guardado en: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\empresas_universe_compilado.csv

	name_norm	name_raw	sources	n_rows
0	3DPHARMA	3dpharma	HORAS	1
1	7 ELEVEN MEXICO	7-ELEVEN MEXICO	LEADS	1
2	ABBOTT	ABBOTT	LEADS	1
3	ABBVIE	Abbvie	LEADS	1
4	ACCO BRANDS	ACCO BRANDS	LEADS	1
5	ACH FOOD COMPANIES	ACH FOOD COMPANIES, INC	LEADS	1
6	ACTINVER	ACTINVER	LEADS	1

7	ADAMANTINE	ADAMANTINE	LEADS	1
8	ADIUM	Adium	HORAS	1
9	AFP GENESIS	AFP Genesis	LEADS	1
10	AIG	AIG	LEADS	1
11	AKROS	Akros	LEADS	1
12	ALCIONE MX	ALCIONE MX	LEADS	1
13	ALIMENTOSALCONSUMIDOR	AlimentosAlConsumidor	LEADS	1
14	ALLIANZ MEXICO	ALLIANZ MÉXICO	LEADS	1
15	ALMEXA	Almexa	HORAS	1
16	ALPER SEGUROS	Alper Seguros	HORAS	1
17	ALPEZZI CHOCOLATE	ALPEZZI CHOCOLATE	LEADS	1
18	ALPINA	Alpina	LEADS	1
19	ALVAREZ BARBA	Alvarez Barba S.A	LEADS	1

```
[20]: # === 9.c Verificación RUC para LEADS vs Super (SCVS) ===
# Salida principal: `leads_ruc_exact` (match exacto) y `leads_ruc_sugerido`
# ↳ (match por similitud para no-matcheados)

from pathlib import Path
import os
import pandas as pd
import numpy as np
import re

# Carpeta con archivos SCVS/Super. La buscamos en ubicaciones típicas.
_CANDIDATE_SUPER_DIRS = [
    Path("data_super_compañias"),
    Path("01_data_ingestion_enrichment") / "data_super_compañias",
]
SUPER_DIR = next((p for p in _CANDIDATE_SUPER_DIRS if p.exists()),
↳ _CANDIDATE_SUPER_DIRS[0])

def _normalize_ruc(value) -> str:
    """Normaliza RUC a solo dígitos. Ecuador típico: 13 dígitos."""
    if value is None:
        return ""
    if isinstance(value, float) and np.isnan(value):
        return ""
    s = str(value).strip()
    if not s:
        return ""
    s = s.replace(".", "")
    s = re.sub(r"\D+", "", s)
    return s
```

```

def _pick_col(cols, patterns):
    cols_u = [str(c).upper() for c in cols]
    for pat in patterns:
        for i, cu in enumerate(cols_u):
            if pat in cu:
                return cols[i]
    return None

def _detect_excel_header_row(fp: Path, max_rows: int = 80) -> int:
    """Detecta la fila de encabezado en Excels tipo reporte (títulos arriba)."""
    try:
        raw = pd.read_excel(fp, header=None, nrows=max_rows)
    except Exception:
        return 0

    best_i = 0
    best_score = -1

    must_have_any = ["RUC", "IDENTIFIC"]
    name_tokens = ["NOMBRE", "RAZON", "RAZÓN", "DENOM", "COMPAÑ"]

    for i in range(len(raw)):
        vals = [v for v in raw.iloc[i].tolist() if str(v).lower() != "nan"]
        if not vals:
            continue
        row_text = " ".join(map(str, vals)).upper()

        score = 0
        score += sum(tok in row_text for tok in must_have_any) * 5
        score += sum(tok in row_text for tok in name_tokens) * 3
        score += ("EXPEDIENTE" in row_text) * 2
        score += ("ESTADO" in row_text) * 1
        score += ("TIPO" in row_text) * 1

        if score > best_score:
            best_score = score
            best_i = i

    return int(best_i)

def _load_super_companies(folder: Path):
    """Carga múltiples archivos de Super/SCVS y arma un catálogo (nombre_norm_
    ↪ -> ruc).

    - Soporta Excels tipo reporte (detecta fila de encabezado).

```

```

- Filtra a RUCs plausibles (13 dígitos) para evitar falsos positivos.
"""
if not folder.exists():
    raise FileNotFoundError(f"No existe la carpeta: {folder}")

files = sorted([p for p in folder.iterdir() if p.suffix.lower() in {".
↪xlsx", ".xls", ".csv"}])
if not files:
    raise FileNotFoundError(f"No se encontraron archivos .xlsx/.xls/.csv en
↪{folder}")

frames = []
for fp in files:
    try:
        if fp.suffix.lower() == ".csv":
            df = pd.read_csv(fp)
        else:
            header_row = _detect_excel_header_row(fp)
            df = pd.read_excel(fp, header=header_row)
    except Exception as e:
        print(f"[SUPER] No se pudo leer {fp.name}: {e}")
        continue

    if df is None or df.empty:
        continue

    # Limpiar nombres de columnas
    df.columns = [str(c).strip() for c in df.columns]

    name_col = _pick_col(
        df.columns,
        patterns=[
            "RAZON", "RAZÓN", "RAZON SOCIAL", "RAZONSOCIAL",
            "NOMBRE", "DENOM", "DENOMIN", "COMPAÑ", "COMPAN",
            "EMPRESA", "INSTITUC", "ORGANIZ",
        ],
    )

    ruc_col = _pick_col(df.columns, patterns=["RUC", "IDENTIFIC",
↪"IDENTIFICACIÓN", "IDENTIFICACION", "CEDULA", "CÉDULA", "CI"])

    if name_col is None:
        obj_cols = [c for c in df.columns if str(df[c].dtype) in ("object",
↪"string")]
        name_col = obj_cols[0] if obj_cols else None

    if ruc_col is None:

```

```

        # fallback: escoger columna con más valores que parecen RUC (13
↪dígitos)
        best = None
        best_score = -1.0
        for c in df.columns:
            s = df[c].map(_normalize_ruc)
            if not hasattr(s, "str"):
                continue
            non_empty = float((s != "").mean())
            len13 = float((s.str.len() == 13).mean())
            score = non_empty + 3.0 * len13
            if score > best_score:
                best_score = score
                best = c
        ruc_col = best

        if name_col is None or ruc_col is None:
            print(f"[SUPER] Saltando {fp.name}: no pude detectar columnas
↪nombre/RUC")
            continue

        tmp = pd.DataFrame({
            "super_source": fp.name,
            "super_name_raw": df[name_col].astype("string"),
            "super_ruc_raw": df[ruc_col],
        })
        tmp["super_name_raw"] = tmp["super_name_raw"].fillna("").map(lambda x:
↪x.strip())
        tmp["super_ruc"] = tmp["super_ruc_raw"].map(_normalize_ruc)

        # Mantener solo RUCs plausibles (Ecuador = 13 dígitos)
        tmp = tmp[tmp["super_ruc"].astype("string").str.len() == 13].copy()

        tmp["super_name_norm"] = tmp["super_name_raw"].
↪map(normalize_company_name)

        # Mantener solo nombres razonables (con letras)
        tmp = tmp[tmp["super_name_norm"].str.contains(r"[A-Z]", regex=True,
↪na=False)].copy()
        tmp = tmp[tmp["super_name_norm"].str.len() >= 3].copy()

        if tmp.empty:
            continue

        frames.append(tmp[["super_source", "super_name_raw", "super_name_norm",
↪"super_ruc"]])

```

```

    if not frames:
        raise ValueError("Se leyeron archivos, pero no se pudo armar un
        ↪catálogo válido (encabezado/RUC/nombres).")

    super_all = pd.concat(frames, ignore_index=True)

    # Si hay duplicados por nombre_norm con distinto ruc, nos quedamos con el
    ↪más frecuente
    super_lookup = (
        super_all
        .groupby(["super_name_norm", "super_ruc"], as_index=False)
        .size()
        .sort_values(["super_name_norm", "size"], ascending=[True, False])
    )
    super_best = super_lookup.drop_duplicates(subset=["super_name_norm"],
    ↪keep="first")["super_name_norm", "super_ruc"]

    return super_all, super_best

if "leads_companies_clean" not in globals():
    raise NameError("No existe `leads_companies_clean`. Ejecuta primero la
    ↪celda 9.a (limpieza LEADS).")

super_all, super_best = _load_super_companies(SUPER_DIR)
print("[SUPER] Carpeta usada:", str(SUPER_DIR))
print("[SUPER] Registros (con nombre y RUC 13d):", int(super_all.shape[0]))
print("[SUPER] Nombres distinct (lookup):", int(super_best.shape[0]))

# Match exacto por nombre normalizado
leads_ruc_exact = (
    leads_companies_clean
    .merge(super_best, left_on="Company_norm", right_on="super_name_norm",
    ↪how="left")
    .drop(columns=["super_name_norm"])
    .rename(columns={"super_ruc": "RUC"})
)

n_total = int(leads_ruc_exact.shape[0])
n_con_ruc = int(leads_ruc_exact["RUC"].notna().sum())
print(f"[LEADS] Total empresas distinct: {n_total}")
print(f"[LEADS] Con RUC (match exacto): {n_con_ruc} ({(n_con_ruc/
    ↪max(n_total,1))*100:.1f}%)")

```

```

# Para aumentar cobertura: sugerencia fuzzy solo para las que NO tienen RUC por
↳match exacto
# Por defecto bajamos a 80 para explorar cobertura (ajustable con
↳SCVS_FUZZY_THRESHOLD)
UMBRALESIMILITUD = int(os.getenv("SCVS_FUZZY_THRESHOLD", "80"))
unmatched = leads_ruc_exact[leads_ruc_exact["RUC"].isna()].copy()

choices = super_best.copy()
choices["name_norm"] = choices["super_name_norm"]
choices = choices[["name_norm", "super_ruc"]].dropna().drop_duplicates().
↳reset_index(drop=True)
choice_names = choices["name_norm"].tolist()

rows = []
try:
    from rapidfuzz import process, fuzz
    for _, r in unmatched.iterrows():
        q = r["Company_norm"]
        if not q:
            continue
        best = process.extractOne(q, choice_names, scorer=fuzz.token_set_ratio)
        if best is None:
            continue
        best_name, score, _ = best
        if float(score) >= UMBRALESIMILITUD:
            ruc = choices.loc[choices["name_norm"] == best_name, "super_ruc"].
↳iloc[0]
            rows.append({
                "Company_raw": r["Company_raw"],
                "Company_norm": q,
                "best_super_name_norm": best_name,
                "score": float(score),
                "RUC": ruc,
            })
    engine = "rapidfuzz"
except Exception:
    from difflib import SequenceMatcher

    def _ratio(a, b):
        return SequenceMatcher(None, a, b).ratio() * 100

    for _, r in unmatched.iterrows():
        q = r["Company_norm"]
        if not q:
            continue
        best_name = None
        best_score = -1.0

```

```

        for cand in choice_names:
            sc = _ratio(q, cand)
            if sc > best_score:
                best_score = sc
                best_name = cand
        if best_name is not None and best_score >= UMBRAL_SIMILITUD:
            ruc = choices.loc[choices["name_norm"] == best_name, "super_ruc"].
            ↪iloc[0]
            rows.append({
                "Company_raw": r["Company_raw"],
                "Company_norm": q,
                "best_super_name_norm": best_name,
                "score": float(best_score),
                "RUC": ruc,
            })
        engine = "difflib"

leads_ruc_sugerido = pd.DataFrame(rows)
if leads_ruc_sugerido.empty:
    leads_ruc_sugerido = pd.DataFrame(columns=["Company_raw", "Company_norm",
        ↪"best_super_name_norm", "score", "RUC"])
else:
    leads_ruc_sugerido = leads_ruc_sugerido.sort_values("score",
        ↪ascending=False).reset_index(drop=True)

print(f"[LEADS] Motor fuzzy: {engine}")
print(f"[LEADS] Sugerencias fuzzy con score >= {UMBRAL_SIMILITUD}:
    ↪{int(leads_ruc_sugerido.shape[0])}")

display(leads_ruc_exact.head(20))
print("\n--- Sugerencias fuzzy (top 20) ---")
display(leads_ruc_sugerido.head(20))

```

```

e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
    warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
    warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no
default style, apply openpyxl's default
    warn("Workbook contains no default style, apply openpyxl's default")
e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\venv\Lib\site-
packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no

```


3	ACCO BRANDS	ACCO BRANDS	NaN
4	ACH FOOD COMPANIES, INC	ACH FOOD COMPANIES	NaN
5	ACTINVER	ACTINVER	NaN
6	ADAMANTINE	ADAMANTINE	NaN
7	AFP Genesis	AFP GENESIS	NaN
8	AIG	AIG	NaN
9	Akros	AKROS	1791148800001
10	ALCIONE MX	ALCIONE MX	NaN
11	AlimentosAlConsumidor	ALIMENTOSALCONSUMIDOR	NaN
12	ALLIANZ MÉXICO	ALLIANZ MEXICO	NaN
13	ALPEZZI CHOCOLATE	ALPEZZI CHOCOLATE	NaN
14	Alpina	ALPINA	NaN
15	Alvarez Barba S.A	ALVAREZ BARBA	1790360741001
16	AMBROSIA	AMBROSIA	NaN
17	ANHEUSER -BUSCH INBEV	ANHEUSER BUSCH INBEV	NaN
18	La Anita	ANITA	NaN
19	APOTEX	APOTEX	NaN

--- Sugerencias fuzzy (top 20) ---

	Company_raw	Company_norm	
	best_super_name_norm score	RUC	
0	ABBOTT	ABBOTT	
	ABBOTT LABORATORIOS ECUADOR 100.0 0990000670001		
1	AIG	AIG	AIG
	METROPOLITANA SEGUROS REASEGUROS 100.0 1790475247001		
2	AFP Genesis	AFP GENESIS	AFP GENESIS
	ADMINISTRADORA FONDOS FIDEICOMISOS 100.0 0991307605001		
3	Alpina	ALPINA	
	ALPINA BEVERAGE ALPINAGUA 100.0 0991371605001		
4	AMBROSIA	AMBROSIA	
	AMBROSIA BIO WINE S B I C 100.0 1191797340001		
5	BACCHUS CONSULTING GROUP	BACCHUS CONSULTING GROUP	
	CONSULTING 100.0 0993372635001		
6	BALL	BALL	
	BALL 9 S 100.0 0993383152001		
7	La Anita	ANITA	COMERCIAL
	IMPORTADORA SANTA ANITA C A IMSANIT 100.0 0990085188001		
8	BAXTER	BAXTER	
	BAXTER ECUADOR 100.0 1791253531001		
9	AXIONLOG	AXIONLOG	
	AXIONLOG ECUADOR 100.0 0992991178001		
10	Asiauto S.A Kia Motors Ecuador	ASIAUTO KIA MOTORS ECUADOR	
	ASIAUTO 100.0 1791754115001		
11	Confiteca	CONFITECA	
	CONFITECA C A 100.0 1790084604001		

12	Directv	DIRECTV	└
↪	DIRECTV ECUADOR C 100.0 1792067782001		
13	Chubb	CHUBB	└
↪	CHUBB SEGUROS ECUADOR 100.0 1790516008001		
14	Claro	CLARO	└
↪	MINERA RIO CLARO C 100.0 0791844366001		
15	CFB	CFB	└
↪	CENTRO FERRETERO BAMBOO CFB S 100.0 1793230370001		
16	CBC	CBC	└
↪	CENTRAL BARBER COMPAN CBC S 100.0 0993388951001		
17	Carval	CARVAL	└
↪	CARVAL ECUADOR CARVALECSA 100.0 0993107859001		
18	CARGILL	CARGILL	└
↪	CARGILL AQUANUTRITION ECUADOR 100.0 0993376610001		
19	Biopas Laboratorios	BIOPAS LABORATORIOS	└
↪	LABORATORIOS BIOPAS 100.0 1791891112001		

```
[21]: # === 9.c.b Verificación RUC para HORAS vs Super (SCVS) ===
# Salida principal: `horas_ruc_exact` (match exacto) y `horas_ruc_sugerido`
↪(match por similitud para no-matcheados)

import os
import pandas as pd
from pathlib import Path

if "horas_empresas_clean" not in globals():
    raise NameError("No existe `horas_empresas_clean`. Ejecuta primero la celda
↪9.b (limpieza HORAS).")

# Asegurar SUPER_DIR aunque esta celda se ejecute primero
if "SUPER_DIR" not in globals():
    _CANDIDATE_SUPER_DIRS = [
        Path("data_super_compañías"),
        Path("01_data_ingestion_enrichment") / "data_super_compañías",
    ]
    SUPER_DIR = next((p for p in _CANDIDATE_SUPER_DIRS if p.exists()),
↪_CANDIDATE_SUPER_DIRS[0])

# Reusar el catálogo ya cargado (si existe); si no, cargarlo
if "super_best" not in globals():
    super_all, super_best = _load_super_companies(SUPER_DIR)

horas_ruc_exact = (
    horas_empresas_clean
    .merge(super_best, left_on="EMPRESA_norm", right_on="super_name_norm",
↪how="left")
    .drop(columns=["super_name_norm"])
```

```

        .rename(columns={"super_ruc": "RUC"})
    )

n_total = int(horas_ruc_exact.shape[0])
n_con_ruc = int(horas_ruc_exact["RUC"].notna().sum())
print(f"[HORAS] Total empresas distinct: {n_total}")
print(f"[HORAS] Con RUC (match exacto): {n_con_ruc} ({(n_con_ruc/
    ↪max(n_total,1))*100:.1f}%)")

# Sugerencias fuzzy solo para los no-matcheados
# Por defecto bajamos a 80 para explorar cobertura (ajustable con
    ↪SCVS_FUZZY_THRESHOLD)
UMBRALESIMILITUD = int(os.getenv("SCVS_FUZZY_THRESHOLD", "80"))
unmatched = horas_ruc_exact[horas_ruc_exact["RUC"].isna()].copy()

choices = super_best.copy()
choices["name_norm"] = choices["super_name_norm"]
choices = choices[["name_norm", "super_ruc"]].dropna().drop_duplicates().
    ↪reset_index(drop=True)
choice_names = choices["name_norm"].tolist()

rows = []
try:
    from rapidfuzz import process, fuzz
    for _, r in unmatched.iterrows():
        q = r["EMPRESA_norm"]
        if not q:
            continue
        best = process.extractOne(q, choice_names, scorer=fuzz.token_set_ratio)
        if best is None:
            continue
        best_name, score, _ = best
        if float(score) >= UMBRALESIMILITUD:
            ruc = choices.loc[choices["name_norm"] == best_name, "super_ruc"].
            ↪iloc[0]
            rows.append({
                "EMPRESA_raw": r["EMPRESA_raw"],
                "EMPRESA_norm": q,
                "best_super_name_norm": best_name,
                "score": float(score),
                "RUC": ruc,
            })
    engine = "rapidfuzz"
except Exception:
    from difflib import SequenceMatcher

    def _ratio(a, b):

```

```

        return SequenceMatcher(None, a, b).ratio() * 100

    for _, r in unmatched.iterrows():
        q = r["EMPRESA_norm"]
        if not q:
            continue
        best_name = None
        best_score = -1.0
        for cand in choice_names:
            sc = _ratio(q, cand)
            if sc > best_score:
                best_score = sc
                best_name = cand
        if best_name is not None and best_score >= UMBRAL_SIMILITUD:
            ruc = choices.loc[choices["name_norm"] == best_name, "super_ruc"].
            ↪iloc[0]
            rows.append({
                "EMPRESA_raw": r["EMPRESA_raw"],
                "EMPRESA_norm": q,
                "best_super_name_norm": best_name,
                "score": float(best_score),
                "RUC": ruc,
            })
        engine = "difflib"

horas_ruc_sugerido = pd.DataFrame(rows)
if horas_ruc_sugerido.empty:
    horas_ruc_sugerido = pd.DataFrame(columns=["EMPRESA_raw", "EMPRESA_norm",
        ↪"best_super_name_norm", "score", "RUC"])
else:
    horas_ruc_sugerido = horas_ruc_sugerido.sort_values("score",
        ↪ascending=False).reset_index(drop=True)

print(f"[HORAS] Motor fuzzy: {engine}")
print(f"[HORAS] Sugerencias fuzzy con score >= {UMBRAL_SIMILITUD}:
    ↪{int(horas_ruc_sugerido.shape[0])}")

display(horas_ruc_exact.head(20))
print("\n--- Sugerencias fuzzy (top 20) ---")
display(horas_ruc_sugerido.head(20))

```

```

[HORAS] Total empresas distinct: 119
[HORAS] Con RUC (match exacto): 18 (15.1%)
[HORAS] Motor fuzzy: rapidfuzz
[HORAS] Sugerencias fuzzy con score >= 80: 72

```

	EMPRESA_raw	EMPRESA_norm	RUC
0	3dpharma	3DPHARMA	NaN

1	Adium	ADIUM	NaN
2	Almexa	ALMEXA	NaN
3	Alper Seguros	ALPER SEGUROS	NaN
4	Arauco	ARAUCO	NaN
5	Aseguradora del Sur	ASEGURADORA SUR	NaN
6	Asesoría y Control	ASESORIA CONTROL	1792765617001
7	AutoShare	AUTOSHARE	NaN
8	AVIS	AVIS	1793119549001
9	BAC	BAC	NaN
10	Banco del Austro	BANCO AUSTR0	NaN
11	Banco Financiero	BANCO FINANCIERO	NaN
12	BANRED	BANRED	0991325026001
13	Bekaert	BEKAERT	NaN
14	Bemis	BEMIS	NaN
15	Bitso	BITS0	NaN
16	Bloomin Brands	BLOOMIN BRANDS	NaN
17	BMI	BMI	NaN
18	Boston BSCI	BOSTON BSCI	NaN
19	CEPSA	CEPSA	1790003388001

--- Sugerencias fuzzy (top 20) ---

	EMPRESA_raw	EMPRESA_norm	
	best_super_name_norm	score	RUC
0	BAC	BAC	
↪	BAC MEDICAL BACME S	100.0	1793212358001
1	Aseguradora del Sur	ASEGURADORA SUR	
↪	ASEGURADORA SUR C A	100.0	0190123626001
2	Corporación Maresa	CORPORACION MARESA	
↪	CORPORACION MARESA HOLDING S	100.0	1791397622001
3	CORSAM	CORSAM	
↪	CORPORACION SAMBORONDON CORSAM	100.0	0992347430001
4	Danec	DANEC	
↪	DANEC ENERGY DANERGY S	100.0	1793236059001
5	CONSEP	CONSEP	CONSTRUCCIONES
↪	SERVICIOS PETROLEROS CONSEP S	100.0	2191774629001
6	Comandato	COMANDATO	COMANDATO
↪	CUENCA SOCIEDAD ANONIMA	100.0	0190002098001
7	BMI	BMI	BMI
↪	ECUADOR SEGUROS VIDA	100.0	1791301692001
8	Unacem	UNACEM	
↪	TRANSPORTES UNACEM UTR	100.0	1793056849001
9	Xcaret	XCARET	
↪	XCARET S	100.0	2490398352001
10	Sunshine	SUNSHINE	NATURE S
↪	SUNSHINE PRODUCTS ECUADOR	100.0	1791308751001

	SIC	SIC	SERVICIOS
11	↪ INTEGRALES CALIFICADOS SIC S	100.0 1891811405001	
12	Tecniseguros	TECNISEGUROS	TECNISEGUROS AGENCIA
	↪ ASESORA PRODUCTORA SEGUROS	100.0 1790048772001	
13	Seguros del Pichincha	SEGUROS PICHINCHA	
	↪ PICHINCHA	100.0 1792893771001	
14	Corporacion GPF	CORPORACION GPF	
	↪ CORPORACION GRUPO FYBECA GPF	100.0 1792287413001	
15	Corona	CORONA	
	↪ CONSTRUCTORA CIVIL CORONA JF TR	100.0 1990928203001	
16	Hospital Metropolitano	HOSPITAL METROPOLITANO	
	↪ HOSPITAL METROPOLITANO S	100.0 1793178138001	
17	Ideal Alambrec Bekaert	IDEAL ALAMBREC BEKAERT	
	↪ IDEAL ALAMBREC	100.0 1790050947001	
18	KPMG	KPMG	CONSORCIO SALOMON SMITH
	↪ BARNEY HAGLER BAILLY K...	100.0 00000000000040	
19	FPA	FPA	FPA CONSTRUCTORA
	↪ INMOBILIARIA S B I C	100.0 0195096309001	

```
[ ]: # === 9.c Carga Centralizada de Catálogos (SCVS y SRI) ===
# Requerimiento: preparar 3 catálogos para torneo (SCVS, SRI Razón Social, SRI
↪ Nombre Fantasía) sin cascadas excluyentes.

from pathlib import Path
import os
import pandas as pd
import numpy as np
import re
import unicodedata

# --- Pre-requisitos ---
if "normalize_company_name" not in globals():
    raise NameError("No existe `normalize_company_name` en memoria. Ejecuta
↪ primero la celda 9.a.")
if "_load_super_companies" not in globals():
    raise NameError("No existe `_load_super_companies`. Ejecuta primero la
↪ verificación SCVS (9.c.a), que define esa función.")

# --- 1) Catálogo SCVS/SUPER ---
if "SUPER_DIR" not in globals():
    _CANDIDATE_SUPER_DIRS = [
        Path("data_super_compañías"),
        Path("01_data_ingestion_enrichment") / "data_super_compañías",
    ]
    SUPER_DIR = next((p for p in _CANDIDATE_SUPER_DIRS if p.exists()),
↪ _CANDIDATE_SUPER_DIRS[0])
```

```

if "super_best" not in globals():
    super_all, super_best = _load_super_companies(SUPER_DIR)

super_best = super_best.copy()
super_best["super_name_norm"] = super_best["super_name_norm"].astype("string")
super_best["super_ruc"] = super_best["super_ruc"].astype("string")
super_choices = super_best["super_name_norm"].dropna().tolist()
super_ruc_by_name = dict(zip(super_best["super_name_norm"].tolist(),
    ↪super_best["super_ruc"].tolist()))
print("[CATALOGOS] SCVS/SUPER - nombres distinct:", int(len(super_choices)))

# --- 2) Catálogos SRI (Razón y Fantasía) ---
SRI_CHUNK_ROWS = int(os.getenv("SRI_CHUNK_ROWS", "200000"))
SRI_DIR = Path(r"E:\TESIS_
    ↪MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\data_SRI")
if not SRI_DIR.exists():
    SRI_DIR = Path("01_data_ingestion_enrichment") / "data_SRI"
if not SRI_DIR.exists():
    raise FileNotFoundError(f"No existe SRI_DIR: {SRI_DIR}")

sri_files = sorted([p for p in SRI_DIR.iterdir() if p.suffix.lower() == ".csv"])
if not sri_files:
    raise FileNotFoundError(f"No hay archivos .csv en: {SRI_DIR}")
print("[CATALOGOS] SRI - archivos CSV detectados:", len(sri_files))

USECOLS_PREFERRED = ["NUMERO_RUC", "RAZON_SOCIAL", "NOMBRE_FANTASIA_COMERCIAL"]

def _normalize_ruc(value) -> str:
    if value is None:
        return ""
    if isinstance(value, float) and np.isnan(value):
        return ""
    s = str(value).strip()
    if not s:
        return ""
    s = s.replace(".0", "")
    s = re.sub(r"\D+", "", s)
    return s

def _sniff_sep(fp: Path) -> str:
    # SRI suele venir con '/' como separador; algunos archivos vienen con otros
    ↪delimitadores.
    try:
        head = fp.open("rb").read(4096)
    except Exception:
        return ", "

```

```

counts = {
    "|": head.count(b"|"),
    ";": head.count(b";"),
    ",": head.count(b","),
    "\t": head.count(b"\t"),
}
sep = max(counts, key=counts.get)
return sep if counts.get(sep, 0) > 0 else ","

def _norm_col(col: str) -> str:
    s = "" if col is None else str(col)
    s = s.strip().upper()
    s = "".join(c for c in unicodedata.normalize("NFKD", s) if not unicodedata.
    combining(c))
    s = re.sub(r"[^A-Z0-9]+", "", s)
    return s

def _pick_sri_columns(fp: Path):
    sep = _sniff_sep(fp)
    encodings_to_try = ["utf-8-sig", "utf-8", "cp1252", "latin1"]

    for enc in encodings_to_try:
        try:
            hdr = pd.read_csv(fp, sep=sep, encoding=enc, dtype=str, nrows=0,
            on_bad_lines="skip")
            cols = list(hdr.columns)
            norm_to_orig = {_norm_col(c): c for c in cols}

            # Alias sets (ya normalizados por _norm_col)
            ruc_aliases = {
                "NUMERORUC",
                "NUMERODERUC",
                "RUC",
            }
            razon_aliases = {
                "RAZONSOCIAL",
                "RAZONSOC",
            }
            fan_aliases = {
                "NOMBREFANTASIACOMERCIAL",
                "NOMBREFANTASIA",
                "NOMBREFANTASIACOMER",
                "NOMBRECOMERCIAL",
            }

            def _find(alias_set):

```



```

        for a in alias_set:
            if a in norm_to_orig:
                return norm_to_orig[a]
        return None

    col_ruc = _find(ruc_alias)
    col_razon = _find(razon_alias)
    col_fan = _find(fan_alias)

    if not col_ruc or not col_razon:
        continue

    return sep, enc, col_ruc, col_razon, col_fan
except Exception:
    continue

raise RuntimeError(
    f"No pude detectar columnas SRI en {fp.name}. "
    f"Esperaba algo equivalente a RUC y RAZON SOCIAL."
)

def _read_sri_chunks(fp: Path):
    # Devuelve un iterador de chunks con columnas estandarizadas:
    # NUMERO_RUC, RAZON_SOCIAL, NOMBRE_FANTASIA_COMERCIAL (puede venir vacía)
    sep, enc, col_ruc, col_razon, col_fan = _pick_sri_columns(fp)

    usecols = [col_ruc, col_razon] + ([col_fan] if col_fan else [])
    chunk_iter = pd.read_csv(
        fp,
        usecols=usecols,
        dtype=str,
        sep=sep,
        chunksize=SRI_CHUNK_ROWS,
        low_memory=True,
        encoding=enc,
        on_bad_lines="skip",
    )

    for chunk in chunk_iter:
        if chunk is None or chunk.empty:
            continue
        rename_map = {
            col_ruc: "NUMERO_RUC",
            col_razon: "RAZON_SOCIAL",
        }
        if col_fan:
            rename_map[col_fan] = "NOMBRE_FANTASIA_COMERCIAL"

```

```

        chunk = chunk.rename(columns=rename_map)
        if "NOMBRE_FANTASIA_COMERCIAL" not in chunk.columns:
            chunk["NOMBRE_FANTASIA_COMERCIAL"] = pd.NA

    yield chunk

# Conteos: (nombre_norm, ruc) -> frecuencia acumulada
count_map_razon = {}
count_map_fan = {}
n_rows_seen = 0

for i, fp in enumerate(sri_files, start=1):
    print(f"[CATALOGOS/SRI] ({i}/{len(sri_files)}) Leyendo: {fp.name}")
    try:
        chunk_iter = _read_sri_chunks(fp)
    except Exception as e:
        print(f"[CATALOGOS/SRI] Saltado: {fp.name} -> {e}")
        continue

    for chunk in chunk_iter:
        if chunk is None or chunk.empty:
            continue
        n_rows_seen += int(len(chunk))

        chunk["NUMERO_RUC"] = chunk["NUMERO_RUC"].map(_normalize_ruc)
        chunk["RAZON_SOCIAL"] = chunk["RAZON_SOCIAL"].astype("string")
        chunk["NOMBRE_FANTASIA_COMERCIAL"] = chunk["NOMBRE_FANTASIA_COMERCIAL"].
        ↪astype("string")

        # Razón social
        tmp = chunk[["NUMERO_RUC", "RAZON_SOCIAL"]].copy()
        tmp = tmp[tmp["RAZON_SOCIAL"].notna() & (tmp["RAZON_SOCIAL"].str.
        ↪strip() != "")]
        if not tmp.empty:
            tmp["name_norm"] = tmp["RAZON_SOCIAL"].map(normalize_company_name)
            tmp = tmp[(tmp["NUMERO_RUC"].str.len() == 13) & (tmp["name_norm"].
            ↪str.len() >= 3)]
            if not tmp.empty:
                grp = tmp.groupby(["name_norm", "NUMERO_RUC"]).size()
                for (name_norm, ruc), cnt in grp.items():
                    key = (str(name_norm), str(ruc))
                    count_map_razon[key] = count_map_razon.get(key, 0) + 1
        ↪int(cnt)

        # Fantasía / comercial (ignorar nulos)
        tmpf = chunk[["NUMERO_RUC", "NOMBRE_FANTASIA_COMERCIAL"]].copy()

```

```

        tmpf = tmpf[tmpf["NOMBRE_FANTASIA_COMERCIAL"].notna() &
        ↪(tmpf["NOMBRE_FANTASIA_COMERCIAL"].str.strip() != "")]
        if not tmpf.empty:
            tmpf["fan_norm"] = tmpf["NOMBRE_FANTASIA_COMERCIAL"].
            ↪map(normalize_company_name)
            tmpf = tmpf[(tmpf["NUMERO_RUC"].str.len() == 13) &
            ↪(tmpf["fan_norm"].str.len() >= 3)]
            if not tmpf.empty:
                grp = tmpf.groupby(["fan_norm", "NUMERO_RUC"]).size()
                for (fan_norm, ruc), cnt in grp.items():
                    key = (str(fan_norm), str(ruc))
                    count_map_fan[key] = count_map_fan.get(key, 0) + int(cnt)

print("[CATALOGOS/SRI] Filas leídas (aprox):", n_rows_seen)
print("[CATALOGOS/SRI] Pares únicos (razon_norm, ruc):", len(count_map_razon))
print("[CATALOGOS/SRI] Pares únicos (fan_norm, ruc):", len(count_map_fan))

if not count_map_razon:
    raise ValueError("No se pudo construir catálogo SRI por Razón Social
    ↪(vacío).")

# Elegir el RUC más frecuente para cada nombre_norm
sri_razon_lookup = (
    pd.DataFrame([{"sri_razon_norm": k[0], "RUC": k[1], "freq": v} for k, v in
    ↪count_map_razon.items()])
    .sort_values(["sri_razon_norm", "freq"], ascending=[True, False])
    .reset_index(drop=True)
)
sri_razon_best = (
    sri_razon_lookup
    .drop_duplicates(subset=["sri_razon_norm"], keep="first")
    [["sri_razon_norm", "RUC"]]
    .reset_index(drop=True)
)
sri_razon_choices = sri_razon_best["sri_razon_norm"].astype("string").dropna().
    ↪tolist()
sri_razon_ruc_by_name = dict(zip(sri_razon_best["sri_razon_norm"].tolist(),
    ↪sri_razon_best["RUC"].astype("string").tolist()))

if count_map_fan:
    sri_fantasia_lookup = (
        pd.DataFrame([{"sri_fantasia_norm": k[0], "RUC": k[1], "freq": v} for
        ↪k, v in count_map_fan.items()])
        .sort_values(["sri_fantasia_norm", "freq"], ascending=[True, False])
        .reset_index(drop=True)
    )

```

```

sri_fantasia_best = (
    sri_fantasia_lookup
    .drop_duplicates(subset=["sri_fantasia_norm"], keep="first")
    [["sri_fantasia_norm", "RUC"]]
    .reset_index(drop=True)
)
else:
    sri_fantasia_best = pd.DataFrame(columns=["sri_fantasia_norm", "RUC"])

sri_fan_choices = sri_fantasia_best["sri_fantasia_norm"].astype("string").
    ↪dropna().tolist()
sri_fan_ruc_by_name = dict(zip(sri_fantasia_best["sri_fantasia_norm"].tolist(),
    ↪sri_fantasia_best["RUC"].astype("string").tolist()))

print("[CATALOGOS] SRI Razón - nombres distinct:", int(len(sri_razon_choices)))
print("[CATALOGOS] SRI Fantasía - nombres distinct:", int(len(sri_fan_choices)))

```

```

[CATALOGOS] SCVS/SUPER - nombres distinct: 214446
[CATALOGOS] SRI - archivos CSV detectados: 26
[CATALOGOS/SRI] (1/26) Leyendo: SRI_Catastro_Empresas_Fantasmas.csv
[CATALOGOS/SRI] (2/26) Leyendo: SRI_MERCADOSENLINEA.csv
[CATALOGOS/SRI] (3/26) Leyendo: SRI_RUC_Azuay.csv
[CATALOGOS/SRI] (4/26) Leyendo: SRI_RUC_Bolivar.csv
[CATALOGOS/SRI] (5/26) Leyendo: SRI_RUC_Carchi.csv
[CATALOGOS/SRI] (6/26) Leyendo: SRI_RUC_Cañar.csv
[CATALOGOS/SRI] (7/26) Leyendo: SRI_RUC_Chimborazo.csv
[CATALOGOS/SRI] (8/26) Leyendo: SRI_RUC_Cotopaxi.csv
[CATALOGOS/SRI] (9/26) Leyendo: SRI_RUC_El_Oro.csv
[CATALOGOS/SRI] (10/26) Leyendo: SRI_RUC_Esmeraldas.csv
[CATALOGOS/SRI] (11/26) Leyendo: SRI_RUC_Galapagos.csv
[CATALOGOS/SRI] (12/26) Leyendo: SRI_RUC_Guayas.csv
[CATALOGOS/SRI] (13/26) Leyendo: SRI_RUC_Imbabura.csv
[CATALOGOS/SRI] (14/26) Leyendo: SRI_RUC_Loja.csv
[CATALOGOS/SRI] (15/26) Leyendo: SRI_RUC_Los_Rios.csv
[CATALOGOS/SRI] (16/26) Leyendo: SRI_RUC_Manabi.csv
[CATALOGOS/SRI] (17/26) Leyendo: SRI_RUC_Morona_Santiago.csv
[CATALOGOS/SRI] (18/26) Leyendo: SRI_RUC_Napo.csv
[CATALOGOS/SRI] (19/26) Leyendo: SRI_RUC_Orellana.csv
[CATALOGOS/SRI] (20/26) Leyendo: SRI_RUC_Pastaza.csv
[CATALOGOS/SRI] (21/26) Leyendo: SRI_RUC_Pichincha.csv

```

```

[ ]: # === 9.d Torneo Fuzzy Matching para LEADS ===
      # Cada empresa huérfana compite simultáneamente en 3 catálogos (SCVS, SRI,
      ↪Razón, SRI Fantasía) y se conserva SOLO el mejor score.

```

```

import os
import pandas as pd

```

```

# --- Entradas requeridas ---
if "leads_ruc_exact" not in globals():
    raise NameError("No existe `leads_ruc_exact`. Ejecuta primero la
↳verificación exacta de RUC para LEADS (SCVS/SUPER).")
if "super_choices" not in globals() or "sri_razon_choices" not in globals() or
↳"sri_fan_choices" not in globals():
    raise NameError("Faltan catálogos/choices. Ejecuta primero la celda 9.c
↳(Carga Centralizada de Catálogos).")
if "super_ruc_by_name" not in globals() or "sri_razon_ruc_by_name" not in
↳globals() or "sri_fan_ruc_by_name" not in globals():
    raise NameError("Faltan mapas nombre->RUC. Ejecuta primero la celda 9.c
↳(Carga Centralizada de Catálogos).")

# Huérfanas = sin match exacto SCVS
unmatched_leads = globals().get("unmatched_leads")
if unmatched_leads is None:
    unmatched_leads = leads_ruc_exact[leads_ruc_exact["RUC"].isna()].copy()
print("[LEADS/TORNEO] Huérfanas (sin RUC por SCVS exacto):",
↳int(unmatched_leads.shape[0]))

UMBRAL_SRI = int(os.getenv("UMBRAL_SRI", "80"))
print("[LEADS/TORNEO] Umbral (UMBRAL_SRI):", UMBRAL_SRI)

try:
    from rapidfuzz import process, fuzz
except Exception as e:
    raise ImportError("Falta rapidfuzz. Instala con: pip install rapidfuzz")
↳from e

# Priorización para desempates (menor = preferido)
_Prio = {"SCVS": 0, "SRI_RAZON": 1, "SRI_FANTASIA": 2}

rows = []
for _, r in unmatched_leads.iterrows():
    q = r.get("Company_norm", "")
    if q is None:
        continue
    q = str(q).strip()
    if not q:
        continue

    contenders = [] # (score, -prio, source, best_name)
    if super_choices:
        b = process.extractOne(q, super_choices, scorer=fuzz.token_set_ratio)
        if b is not None:

```

```

        name, score, _ = b
        contenders.append((float(score), -_PRIO["SCVS"], "SCVS", str(name)))
    if sri_razon_choices:
        b = process.extractOne(q, sri_razon_choices, scorer=fuzz.
↪token_set_ratio)
        if b is not None:
            name, score, _ = b
            contenders.append((float(score), -_PRIO["SRI_RAZON"], "SRI_RAZON",
↪str(name)))
    if sri_fan_choices:
        b = process.extractOne(q, sri_fan_choices, scorer=fuzz.token_set_ratio)
        if b is not None:
            name, score, _ = b
            contenders.append((float(score), -_PRIO["SRI_FANTASIA"],
↪"SRI_FANTASIA", str(name)))

    if not contenders:
        continue

    best_score, _, source_winner, best_name = max(contenders, key=lambda t:
↪(t[0], t[1]))
    if float(best_score) < float(UMBRAL_SRI):
        continue

    if source_winner == "SCVS":
        ruc = super_ruc_by_name.get(best_name, "")
    elif source_winner == "SRI_RAZON":
        ruc = sri_razon_ruc_by_name.get(best_name, "")
    else:
        ruc = sri_fan_ruc_by_name.get(best_name, "")

    rows.append({
        "Company_raw": r.get("Company_raw", ""),
        "Company_norm": q,
        "source_winner": source_winner,
        "winner_name_norm": best_name,
        "score": float(best_score),
        "RUC": "" if ruc is None else str(ruc),
    })

leads_torneo_ganadores = pd.DataFrame(rows)
if leads_torneo_ganadores.empty:
    leads_torneo_ganadores = pd.DataFrame(columns=["Company_raw",
↪"Company_norm", "source_winner", "winner_name_norm", "score", "RUC"])
else:
    leads_torneo_ganadores = leads_torneo_ganadores.sort_values("score",
↪ascending=False).reset_index(drop=True)

```

```

print("[LEADS/TORNEO] Ganadores (score >= umbral):", int(leads_torneo_ganadores.
↳shape[0]))
print("[LEADS/TORNEO] Ganadores por origen:")
print(leads_torneo_ganadores["source_winner"].value_counts(dropna=False).
↳to_string())
display(leads_torneo_ganadores.head(20))

```

```

[ ]: # === 9.e Torneo Fuzzy Matching para HORAS ===
# Misma lógica del torneo de LEADS, pero iterando sobre las empresas huérfanas
↳de HORAS.

import os
import pandas as pd

# --- Entradas requeridas ---
if "horas_ruc_exact" not in globals():
    raise NameError("No existe `horas_ruc_exact`. Ejecuta primero la
↳verificación exacta de RUC para HORAS (SCVS/SUPER).")
if "super_choices" not in globals() or "sri_razon_choices" not in globals() or
↳"sri_fan_choices" not in globals():
    raise NameError("Faltan catálogos/choices. Ejecuta primero la celda 9.c
↳(Carga Centralizada de Catálogos).")
if "super_ruc_by_name" not in globals() or "sri_razon_ruc_by_name" not in
↳globals() or "sri_fan_ruc_by_name" not in globals():
    raise NameError("Faltan mapas nombre->RUC. Ejecuta primero la celda 9.c
↳(Carga Centralizada de Catálogos).")

# Huérfanas = sin match exacto SCVS
unmatched_horas = globals().get("unmatched_horas")
if unmatched_horas is None:
    unmatched_horas = horas_ruc_exact[horas_ruc_exact["RUC"].isna()].copy()
print("[HORAS/TORNEO] Huérfanas (sin RUC por SCVS exacto):",
↳int(unmatched_horas.shape[0]))

UMBRAL_SRI = int(os.getenv("UMBRAL_SRI", "80"))
print("[HORAS/TORNEO] Umbral (UMBRAL_SRI):", UMBRAL_SRI)

try:
    from rapidfuzz import process, fuzz
except Exception as e:
    raise ImportError("Falta rapidfuzz. Instala con: pip install rapidfuzz")
↳from e

# Priorización para desempates (menor = preferido)
_PRIO = {"SCVS": 0, "SRI_RAZON": 1, "SRI_FANTASIA": 2}

```

```

rows = []
for _, r in unmatched_horas.iterrows():
    q = r.get("EMPRESA_norm", "")
    if q is None:
        continue
    q = str(q).strip()
    if not q:
        continue

    contenders = [] # (score, -prio, source, best_name)
    if super_choices:
        b = process.extractOne(q, super_choices, scorer=fuzz.token_set_ratio)
        if b is not None:
            name, score, _ = b
            contenders.append((float(score), -_PRIO["SCVS"], "SCVS", str(name)))
    if sri_razon_choices:
        b = process.extractOne(q, sri_razon_choices, scorer=fuzz.
↪token_set_ratio)
        if b is not None:
            name, score, _ = b
            contenders.append((float(score), -_PRIO["SRI_RAZON"], "SRI_RAZON",
↪str(name)))
    if sri_fan_choices:
        b = process.extractOne(q, sri_fan_choices, scorer=fuzz.token_set_ratio)
        if b is not None:
            name, score, _ = b
            contenders.append((float(score), -_PRIO["SRI_FANTASIA"],
↪"SRI_FANTASIA", str(name)))

    if not contenders:
        continue

    best_score, _, source_winner, best_name = max(contenders, key=lambda t:
↪t[0], t[1])
    if float(best_score) < float(UMBRAL_SRI):
        continue

    if source_winner == "SCVS":
        ruc = super_ruc_by_name.get(best_name, "")
    elif source_winner == "SRI_RAZON":
        ruc = sri_razon_ruc_by_name.get(best_name, "")
    else:
        ruc = sri_fan_ruc_by_name.get(best_name, "")

    rows.append({
        "EMPRESA_raw": r.get("EMPRESA_raw", ""),

```



```

        "EMPRESA_norm": q,
        "source_winner": source_winner,
        "winner_name_norm": best_name,
        "score": float(best_score),
        "RUC": "" if ruc is None else str(ruc),
    })

horas_torneo_ganadores = pd.DataFrame(rows)
if horas_torneo_ganadores.empty:
    horas_torneo_ganadores = pd.DataFrame(columns=["EMPRESA_raw",
    ↪ "EMPRESA_norm", "source_winner", "winner_name_norm", "score", "RUC"])
else:
    horas_torneo_ganadores = horas_torneo_ganadores.sort_values("score",
    ↪ ascending=False).reset_index(drop=True)

print("[HORAS/TORNEO] Ganadores (score >= umbral):", int(horas_torneo_ganadores.
    ↪ shape[0]))
print("[HORAS/TORNEO] Ganadores por origen:")
print(horas_torneo_ganadores["source_winner"].value_counts(dropna=False).
    ↪ to_string())
display(horas_torneo_ganadores.head(20))

```

```

[ ]: # === 9.f Preparar auditoría LLM con los ganadores ===
# Requerimiento: armar `audit_cases` EXCLUSIVAMENTE desde
    ↪ `leads_torneo_ganadores` y `horas_torneo_ganadores` y reportar métricas por
    ↪ origen.

import os
import json
import pandas as pd

if "leads_torneo_ganadores" not in globals():
    raise NameError("No existe `leads_torneo_ganadores`. Ejecuta primero 9.d.")
if "horas_torneo_ganadores" not in globals():
    raise NameError("No existe `horas_torneo_ganadores`. Ejecuta primero 9.e.")

# Para ahorrar tokens/costo: recorta textos muy largos
MAX_NAME_CHARS = int(os.getenv("AUDIT_MAX_NAME_CHARS", "160"))
def _clip_text(x, n: int = MAX_NAME_CHARS) -> str:
    if x is None or (isinstance(x, float) and pd.isna(x)):
        return ""
    s = str(x)
    return s if len(s) <= n else (s[:n] + "...")

AUDIT_THRESHOLD = int(os.getenv("UMBRAL_SRI", "80"))

```

```

# Prompt de auditoría (compacto). Regla clave: NO inventar; decidir solo con
↳ los textos entregados.
LLM_JUDGE_PROMPT = """Eres auditor de matching de empresas (Ecuador).
Entrada: JSON {cases:[{id,source_raw,source_norm,candidate_norm,ruc,score}]}.
Decide si el match es plausible SOLO con esos textos (sin conocimiento externo).

Reglas:
- Si ruc no tiene 13 dígitos => verdict=incorrect (confidence alta).
- Si comparten tokens distintivos y no hay conflicto => correct.
- Si es genérico, corto o evidencia insuficiente => uncertain.

Salida: JSON con clave results = lista de objetos {id, verdict, confidence,
↳ reason}.
confidence en [0,1]. reason: 1 frase corta (<= 80 chars), sin saltos de línea y
↳ SIN comillas dobles (usa comillas simples si hace falta)."""

def _build_cases_from_winners(df: pd.DataFrame, prefix: str) -> list:
    if df is None or getattr(df, "empty", True):
        return []
    d = df.copy()
    if "score" in d.columns:
        d = d[d["score"].astype(float) >= float(AUDIT_THRESHOLD)].copy()
    if d.empty:
        return []
    d = d.sort_values("score", ascending=False).reset_index(drop=True)

    raw_col = "Company_raw" if "Company_raw" in d.columns else ("EMPRESA_raw"
↳ if "EMPRESA_raw" in d.columns else None)
    norm_col = "Company_norm" if "Company_norm" in d.columns else
↳ ("EMPRESA_norm" if "EMPRESA_norm" in d.columns else None)
    cand_col = "winner_name_norm" if "winner_name_norm" in d.columns else None
    ruc_col = "RUC" if "RUC" in d.columns else None

    cases = []
    for i, r in d.iterrows():
        source_winner = str(r.get("source_winner", ""))
        case_id = f"{prefix}-{source_winner}-{i}"
        cases.append({
            "id": case_id,
            "source_label": f"{prefix}_{source_winner}",
            "source_raw": _clip_text(r.get(raw_col, "") if raw_col else ""),
            "source_norm": _clip_text(r.get(norm_col, "") if norm_col else ""),
            "candidate_norm": _clip_text(r.get(cand_col, "") if cand_col else
↳ ""),
            "score": None if pd.isna(r.get("score")) else float(r.get("score")),
            "ruc": "" if pd.isna(r.get(ruc_col)) else str(r.get(ruc_col)),

```

```

    })
    return cases

audit_cases = []
audit_cases += _build_cases_from_winners(leads_torneo_ganadores, "LEADS")
audit_cases += _build_cases_from_winners(horas_torneo_ganadores, "HORAS")

audit_df = pd.DataFrame(audit_cases)

# --- Métricas claras por origen (ganadores) ---
print("[AUDITORIA] Umbral (UMBRAL_SRI):", AUDIT_THRESHOLD)

print("\n[LEADS] Ganadores totales:", int(leads_torneo_ganadores.shape[0]))
print("[LEADS] Ganadores por origen:")
print(leads_torneo_ganadores["source_winner"].value_counts(dropna=False).
      ↪to_string())

print("\n[HORAS] Ganadores totales:", int(horas_torneo_ganadores.shape[0]))
print("[HORAS] Ganadores por origen:")
print(horas_torneo_ganadores["source_winner"].value_counts(dropna=False).
      ↪to_string())

counts = audit_df.groupby("source_label").size().to_dict() if not audit_df.
      ↪empty else {}
print("\n[AUDITORIA] Casos a auditar (armados) total:", len(audit_cases))
print("[AUDITORIA] Casos a auditar por fuente:", counts)
print("Ejemplo (1 caso):")
print(json.dumps(audit_cases[:1], ensure_ascii=False, indent=2))

```

Umbral SUPER (SCVS_FUZZY_THRESHOLD): 80

Umbral SRI (UMBRAL_SRI): 80

Sugerencias fuzzy disponibles: LEADS_SUPER=181 , HORAS_SUPER=72 ,
 LEADS_SRI_RAZON=12 , HORAS_SRI_RAZON=10 , LEADS_SRI_FANTASIA=0 ,
 HORAS_SRI_FANTASIA=0

Casos a auditar (armados) total: 275

Casos a auditar por fuente: {'HORAS_SRI_RAZON': 10, 'HORAS_SUPER': 72,
 'LEADS_SRI_RAZON': 12, 'LEADS_SUPER': 181}

Ejemplo (1 caso):

```

[
  {
    "id": "LEADS_SUPER-0",
    "source_label": "LEADS_SUPER",
    "source_raw": "ABBOTT",
    "candidate_norm": "ABBOTT LABORATORIOS ECUADOR",
    "score": 100.0,
    "ruc": "0990000670001"
  }
]

```

]

```
[ ]: # === 9.g (Opcional) Llamar API (ChatGPT/OpenAI) para auditar matches fuzzy ===  
# Este paso manda TODOS los casos de `audit_cases` (celda 9.f) a la API, en  
  ↳ chunks para evitar truncados.  
  
import os  
import json  
import urllib.request  
import urllib.error  
import pandas as pd  
  
if "audit_cases" not in globals():  
    raise NameError("No existe `audit_cases`. Ejecuta primero la celda 9.f.")  
  
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")  
if not OPENAI_API_KEY:  
    try:  
        import getpass  
        OPENAI_API_KEY = getpass.getpass("Ingresa OPENAI_API_KEY (entrada  
  ↳ oculta, no se guarda): ").strip()  
    except Exception:  
        OPENAI_API_KEY = None  
  
if not OPENAI_API_KEY:  
    raise EnvironmentError(  
        "No se encontró `OPENAI_API_KEY`. "  
        "Configúrala como variable de entorno o ingrésala cuando se solicite."  
    )  
  
OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")  
OPENAI_URL = os.getenv("OPENAI_URL", "https://api.openai.com/v1/chat/  
  ↳ completions")  
  
def _extract_json_object(text: str) -> str:  
    if text is None:  
        return ""  
    s = str(text).strip()  
    if s.startswith("`"):   
        s = s.split("\n", 1)[1]  
        s = s.rsplit("`", 1)[0].strip()  
    i = s.find("{")  
    j = s.rfind("}")  
    return s[i:j+1] if i != -1 and j != -1 and j > i else s
```

```

def _call_openai_judge(cases_chunk: list) -> dict:
    payload = {
        "model": OPENAI_MODEL,
        "temperature": 0,
        # salida por chunk (ajústalo si ves truncado)
        "max_tokens": int(os.getenv("OPENAI_MAX_TOKENS", "2000")),
        "response_format": {"type": "json_object"},
        "messages": [
            {"role": "system", "content": LLM_JUDGE_PROMPT},
            {"role": "user", "content": json.dumps({"cases": cases_chunk},
↪ensure_ascii=False)},
        ],
    }

    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        OPENAI_URL,
        data=data,
        headers={"Content-Type": "application/json", "Authorization": f"Bearer_↪
↪{OPENAI_API_KEY}"},
        method="POST",
    )

    try:
        with urllib.request.urlopen(req, timeout=300) as resp:
            raw = resp.read().decode("utf-8")
    except urllib.error.HTTPError as e:
        err = e.read().decode("utf-8", errors="ignore")
        raise RuntimeError(f"HTTPError {e.code}: {err}")

    out = json.loads(raw)
    content = out["choices"][0]["message"]["content"]
    content = _extract_json_object(content)
    return json.loads(content)

def _judge_in_chunks(cases_all: list) -> dict:
    if not cases_all:
        return {"results": []}

    chunk_size = int(os.getenv("AUDIT_CHUNK_SIZE", "10"))
    min_chunk = int(os.getenv("AUDIT_MIN_CHUNK", "2"))
    results_all = []

    i = 0
    while i < len(cases_all):
        cur = cases_all[i:i+chunk_size]

```

```

    try:
        out = _call_openai_judge(cur)
        res = out.get("results", [])
        if not isinstance(res, list):
            raise ValueError("Respuesta sin results-list")
        results_all.extend(res)
        i += chunk_size
    except (json.JSONDecodeError, ValueError) as e:
        if chunk_size <= min_chunk:
            raise RuntimeError(f"No pude parsear JSON con_
↳ chunk_size={chunk_size}. Error: {e}") from e
        chunk_size = max(min_chunk, chunk_size // 2)
        continue

    return {"results": results_all}

print("Casos a enviar a IA:", len(audit_cases))
judge_json = _judge_in_chunks(list(audit_cases))
results = judge_json.get("results", [])

judge_df = pd.DataFrame(results)
if judge_df.empty:
    raise ValueError("La API respondió, pero no devolvió `results`. Revisa el_
↳ JSON retornado.")

cases_df = pd.DataFrame(audit_cases)
out_df = cases_df.merge(judge_df, on="id", how="left")

print("Resultados auditados:", len(out_df))
print(out_df["verdict"].value_counts(dropna=False))

cols = ["source_label", "source_raw", "candidate_norm", "score", "ruc",_
↳ "verdict", "confidence", "reason"]
cols = [c for c in cols if c in out_df.columns]
display(out_df.sort_values(["source_label", "score"], ascending=[True,_
↳ False])[cols].head(120))

```

```

-----
OSError                                Traceback (most recent call last)
Cell In[66], line 22
     18     except Exception:
     19         OPENAI_API_KEY = None
     20
     21 if not OPENAI_API_KEY:
--> 22     raise EnvironmentError(
     23         "No se encontró `OPENAI_API_KEY`. "

```

```

24         "Configúrala como variable de entorno o ingrésala cuando se
↪solicite."
25     )

```

`OSError`: No se encontró ``OPENAI_API_KEY``. Configúrala como variable de entorno o ingrésala cuando se solicite.

```

[ ]: # === Export final a CSV (salida de la última celda) ===
# Esta celda guarda a disco el DataFrame final (por defecto: `out_df`) para que
↪lo puedas descargar desde VS Code.

from pathlib import Path
from datetime import datetime
import pandas as pd

# 1) Detectar el DataFrame final (prioridad: out_df)
df_final = None
df_name = None

if "out_df" in globals() and isinstance(globals().get("out_df"), pd.DataFrame):
    df_final = globals()["out_df"]
    df_name = "out_df"
elif "audit_df" in globals() and isinstance(globals().get("audit_df"), pd.
↪DataFrame):
    df_final = globals()["audit_df"]
    df_name = "audit_df"
else:
    # fallback: intenta con el último df "razonable" en memoria
    for cand in ["leads_sri_sugerido", "horas_sri_sugerido",
↪"leads_ruc_sugerido", "horas_ruc_sugerido"]:
        if cand in globals() and isinstance(globals().get(cand), pd.DataFrame):
            df_final = globals()[cand]
            df_name = cand
            break

if df_final is None:
    raise NameError("No encontré un DataFrame final para exportar (esperaba
↪`out_df`). Ejecuta la última celda y vuelve a intentar.")
if df_final.empty:
    raise ValueError(f"El DataFrame `{df_name}` está vacío; no exporto un CSV
↪vacío.")

# 2) Ruta de salida robusta (si el cwd ya es 01_data_ingestion_enrichment, usa
↪outputs/)
OUT_DIR = Path("outputs") if Path.cwd().name == "01_data_ingestion_enrichment"
↪else (Path("01_data_ingestion_enrichment") / "outputs")

```

```

OUT_DIR.mkdir(parents=True, exist_ok=True)

stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
out_path = OUT_DIR / f"{df_name}_export_{stamp}.csv"

# 3) Guardar
df_final.to_csv(out_path, index=False, encoding="utf-8-sig")
print(f"CSV exportado desde `{df_name}` -> {out_path.resolve()}")
print("Filas:", len(df_final), "| Columnas:", df_final.shape[1])

```

CSV exportado desde `out\_df` -> E:\TESIS MAESTRIA\Desarrollo_clustering_maestria
 \01_data_ingestion_enrichment\outputs\out_df_export_20260412_163051.csv
 Filas: 275 | Columnas: 9